NEXUS

GDI Client Operating System

Engineering Documentation

PT Global Dataverse Indonesia

Version 1.0

April 2026

CONFIDENTIAL — INTERNAL USE ONLY

Table of Contents

Table of Contents 2
1. Introduction 4
1.1 Purpose 4
1.2 Design Principles 4
2. System Architecture 5
2.1 Architecture Layers 5
Frontend (Nexus UI) 5
API Layer 5
Backend Services 5
2.2 External Integrations 5
2.3 Tech Stack Summary 6
3. Client Lifecycle Flow 7
3.1 Lifecycle Stages 7
Stage 1: AI Discovery 7
Stage 2: Portal Registration & Lead Submission 7
Stage 3: Scheduling 7
Stage 4: Consultation & Decision 7
Stage 5: Pre-Development & NDA 7
Stage 6: Development & PoC Deployment 7
Stage 7: Payment Model Selection 7
Stage 8: Post-Delivery & Retention 8
4. Module Interaction Map 9
4.1 Core Entity Relationships 9
4.2 Cross-Module Dependencies 9
5. State System 10
5.1 Client States 10
5.2 Project States 10
5.3 Request States 10
6. Core Request Lifecycle 11
6.1 Flow Description 11
6.2 Request Categories 11
6.3 Priority Levels 11
7. Notification Flow 12
7.1 How It Works 12
7.2 Trigger Events 12
7.3 Implementation Notes 12

8. User Roles & Permissions 13
8.1 External Roles 13
Visitor (No Login) 13
Prospect (Registered, Not Yet Qualified) 13
Client (Qualified & Active) 13
8.2 Internal Roles 13
Engineer 13
Admin / GDI 13
8.3 Permission Matrix 13
9. Core Modules Reference 15
9.1 AI Intake Module 15
9.2 Lead Management Module 15
9.3 Scheduling Module 15
9.4 Client Workspace Module 15
9.5 Ticketing System 15
9.6 Contract Module 15
9.7 Billing Module 16
9.8 Monitoring Module (Phase 3) 16
10. Client Types 17
11. Phased Development Plan 18
Phase 1 — MVP: Structured Lead Acquisition 18
Phase 2 — Client Operations 18
Phase 3 — Deal Closure 18
Phase 4 — Scale & Differentiate 18
12. Key Engineering Guidelines 19
12.1 General Principles 19
12.2 API Design 19
12.3 Database Conventions 19
12.4 Notification Conventions 19

1. Introduction

This document serves as the primary engineering reference for Nexus, GDI's Client Operating System (GDI-COS). It is intended for the development team and covers the system's architecture, module structure, data flows, state management, and phased build plan.

Nexus is a unified digital platform that manages the entire lifecycle of client engagement at PT Global Dataverse Indonesia, from initial discovery through project delivery, billing, and long-term support.

1.1 Purpose

Nexus consolidates all client-facing and internal operational processes into a single platform. Rather than relying on fragmented tools for CRM, project management, invoicing, and support, the engineering team will build one coherent system that handles every stage.

1.2 Design Principles

- **One identity per client:** A single client record persists across all lifecycle stages. No duplicate accounts, no fragmented profiles.
- **Continuous lifecycle:** Clients move through stages fluidly without switching between disconnected systems.
- **Automation supports decisions:** The system automates scheduling, notifications, and status tracking, but critical business decisions remain human-driven.
- **Clean client UX, powerful internal backend:** Clients see a simple dashboard. GDI staff see full operational controls.
- **Progressive enhancement:** Build MVP first, then iterate. Each phase adds capability without breaking existing functionality.

2. System Architecture

Nexus follows a layered architecture with a Next.js frontend, an API layer handling all business logic, and multiple backend services for specific domains.

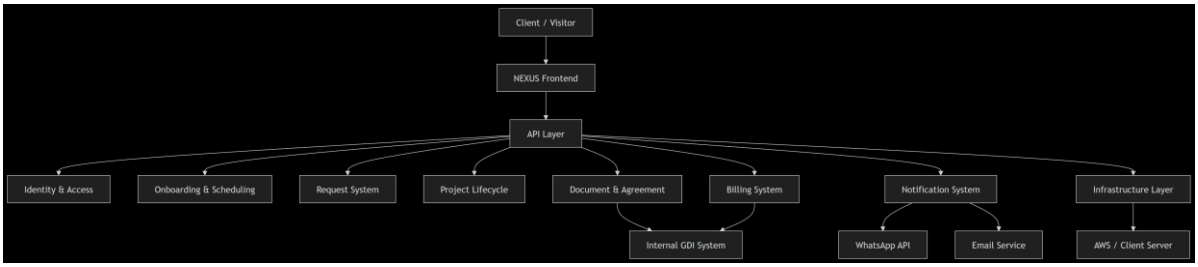

Figure 1 — System Architecture Overview

2.1 Architecture Layers

Frontend (Nexus UI)

- Built on Next.js (existing GDI stack)
- Serves both public-facing pages (AI intake, scheduling) and authenticated dashboards (client workspace, admin panels)
- Communicates exclusively through the API layer

API Layer

Central gateway that routes requests to the appropriate backend service. All client and internal interactions pass through this layer. The API layer handles authentication, authorization, request validation, and routing.

Backend Services

The API layer connects to seven core backend services:

Service	Responsibility
Identity & Access	Authentication, authorization, role management, session handling
Onboarding & Scheduling	Calendar sync, meeting booking, availability management
Request System	Ticket creation, assignment, status tracking, priority management
Project Lifecycle	Project states, milestones, task tracking, deliverable management
Document & Agreement	Contract templates, versioning, e-signature integration
Billing System	Invoices, subscriptions, payment gateway integration
Notification System	WhatsApp API and email dispatch for all system events
Infrastructure Layer	Server monitoring, uptime tracking, health dashboards (Phase 3)

2.2 External Integrations

- **Calendar API:** Google Calendar / Outlook for scheduling sync
- **Payment Gateway:** Stripe or local Indonesian gateway (e.g., Midtrans)

- **E-Signature:** DocuSign or Adobe Acrobat Sign for contract execution
- **WhatsApp Business API:** Primary notification channel
- **Email Service:** Fallback notification channel
- **AWS / Client Server:** Infrastructure monitoring targets

2.3 Tech Stack Summary

Layer	Technology
Frontend	Next.js (React), TypeScript
Backend / API	Next.js API Routes or Node.js
Database	PostgreSQL (primary) / Firebase (optional)
Auth	JWT / session-based
Hosting	Cloud-based (AWS / GCP / VPS)

3. Client Lifecycle Flow

The client lifecycle defines the end-to-end journey from initial contact to long-term retention. Every stage maps to specific system modules and state transitions.

Figure 2 — Client Lifecycle Flow

3.1 Lifecycle Stages

Stage 1: AI Discovery

A visitor interacts with the AI intake agent on the GDI website. No login is required. The agent structures the visitor's business needs into a problem summary, service classification, and preliminary scope.

Stage 2: Portal Registration & Lead Submission

The visitor submits their request, optionally creating an account for tracking. The system captures company profile, problem description, service category, urgency level, estimated budget, and preferred timeline.

Stage 3: Scheduling

The prospect selects a meeting type (online/offline) and preferred time. The system syncs with GDI's calendar, shows available slots, and confirms booking automatically. Notifications are sent to both parties.

Stage 4: Consultation & Decision

After the consultation meeting, the internal team makes a qualification decision. This is a critical decision point with three outcomes:

- **Proceed:** Client is qualified. Account is created (if not existing) and the client enters the workspace.
- **Delay:** The account is frozen. The prospect remains in the system for future follow-up.
- **Reject:** Lead is archived with notes.

Stage 5: Pre-Development & NDA

Qualified clients receive access to the client workspace. NDA and preliminary documents are exchanged. The project scope is formally defined before development begins.

Stage 6: Development & PoC Deployment

Active development tracked through the project lifecycle module. Milestones, tasks, and deliverables are visible to the client. Upon completion, a Proof of Concept is deployed for validation.

Stage 7: Payment Model Selection

Based on the engagement type, the client selects a payment model:

- **Subscription:** Recurring payments for cloud-deployed systems with active support and maintenance.
- **One-Time:** Single payment for client-deployed systems.

Stage 8: Post-Delivery & Retention

After delivery, the client dashboard remains active. Clients can submit support tickets, request maintenance, and access all historical documents. This stage feeds the long-term retention loop.

4. Module Interaction Map

This diagram shows how the core data entities and modules relate to each other within Nexus.

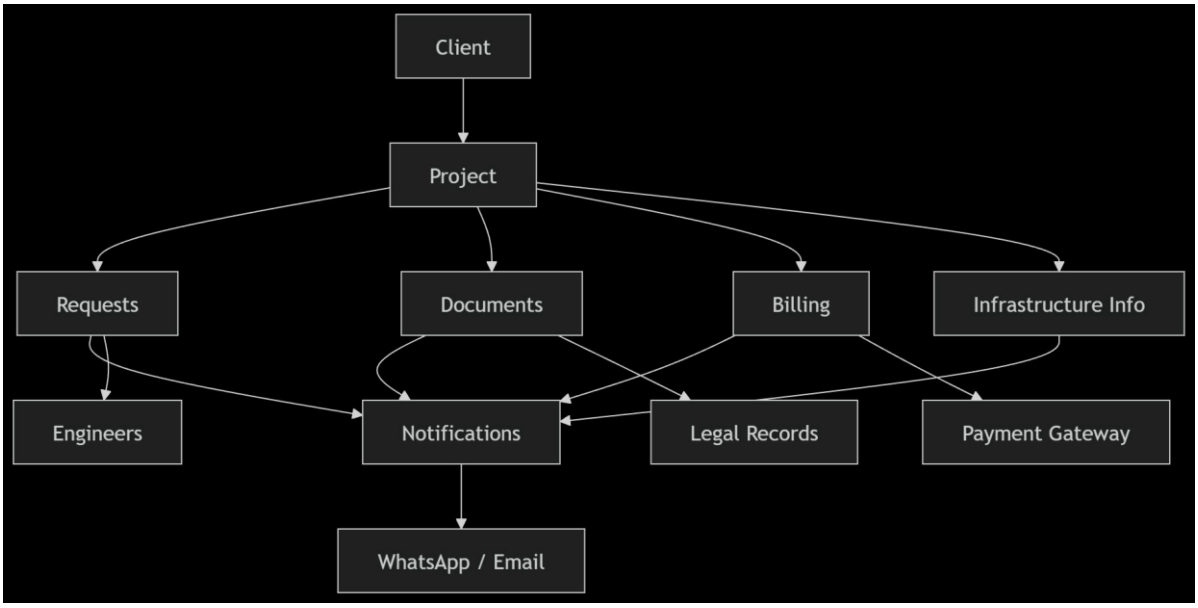

Figure 3 — Module Interaction Map

4.1 Core Entity Relationships

The Client entity sits at the top of the hierarchy. Each client has one or more Projects. Each Project connects to four primary modules:

- **Requests:** Support tickets and change requests, assigned to Engineers for resolution.
- **Documents:** Contracts, deliverables, and shared files. Connected to Legal Records for agreement management.
- **Billing:** Invoices and payment tracking, connected to the Payment Gateway and Legal Records.
- **Infrastructure Info:** Server health, uptime, and monitoring data (Phase 3).

4.2 Cross-Module Dependencies

The Notifications module acts as a central hub, receiving events from Requests, Documents, and Billing. All notifications are dispatched via WhatsApp (primary) or Email (fallback). Engineers receive assignment notifications. Clients receive status updates, document alerts, and billing reminders.

5. State System

Nexus uses a structured state machine across three primary entities. States drive UI visibility, available actions, and internal workflows.

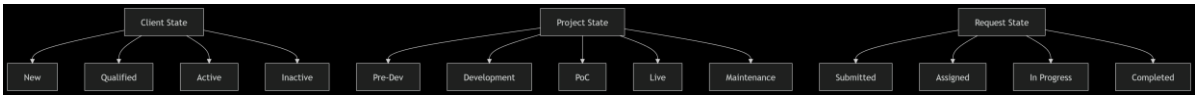

Figure 4 — State System

5.1 Client States

State	Description	Triggered By
New	Lead just submitted, awaiting review	Form submission or AI intake
Qualified	Passed consultation, approved for engagement	Internal decision (proceed)
Active	Has at least one active project	Project creation
Inactive	No active projects, post-delivery or frozen	Project completion or delay decision

5.2 Project States

State	Description
Pre-Dev	Scope defined, NDA signed, awaiting kickoff
Development	Active build phase with milestone tracking
PoC	Proof of Concept deployed for client validation
Live	Production deployment complete
Maintenance	Post-delivery support and monitoring active

5.3 Request States

State	Description
Submitted	Client has submitted a request, awaiting assignment
Assigned	Request assigned to an engineer
In Progress	Engineer actively working on the request
Completed	Work finished, client notified

Important: State transitions should be enforced at the API layer, not just the frontend. Invalid transitions (e.g., jumping from “New” directly to “Active”) must be rejected.

6. Core Request Lifecycle

The request lifecycle defines how support tickets and change requests flow through the system from submission to resolution.

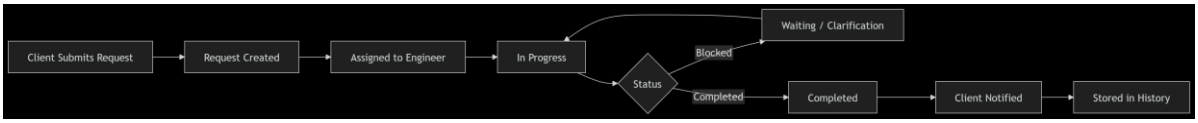

Figure 5 — Core Request Lifecycle

6.1 Flow Description

When a client submits a request, the system creates a request record and assigns it to an engineer (automatically or manually by admin). The engineer works on the request, updating its status to “In Progress.”

At any point during progress, the request may hit a blocker requiring client clarification. In this case, the status moves to “Waiting / Clarification” and the client is notified. Once clarification is received, the request returns to “In Progress.”

Upon completion, the client is notified and the request is stored in history for audit and reference.

6.2 Request Categories

- Bug / Error
- Access Issue
- Feature Request
- Maintenance
- Billing Issue

6.3 Priority Levels

Priority	Response Target	Use Case
Critical	< 2 hours	System down, data loss, security breach
High	< 8 hours	Major feature broken, blocking client operations
Medium	< 24 hours	Non-critical bug, performance issue
Low	< 72 hours	Cosmetic issue, feature request, general inquiry

7. Notification Flow

Nexus uses a centralized notification engine that dispatches messages through WhatsApp (primary) with email as fallback.

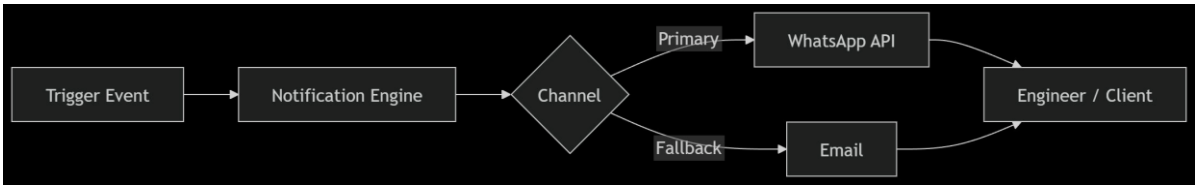

Figure 6 — Notification Flow (WhatsApp + Email)

7.1 How It Works

A trigger event (e.g., request status change, invoice generated, meeting confirmed) fires into the Notification Engine. The engine determines the appropriate channel:

- **Primary — WhatsApp API:** Used for time-sensitive notifications. Requires client to have a registered WhatsApp number.
- **Fallback — Email:** Used when WhatsApp delivery fails or when the notification type requires a longer format (e.g., contract documents).

7.2 Trigger Events

Event	Recipients	Channel Priority
New request submitted	Assigned engineer + admin	WhatsApp
Request status updated	Client	WhatsApp
Meeting scheduled	Client + consultant	WhatsApp + Email
Invoice generated	Client	Email (with PDF)
Contract ready for signature	Client	Email
System health alert	Admin + engineer	WhatsApp
Payment received	Client + finance	Email

7.3 Implementation Notes

- All notification templates should be version-controlled and stored in the database
- WhatsApp messages must comply with Meta's Business API template policies
- Implement retry logic with exponential backoff for failed deliveries
- Log all notification attempts with delivery status for debugging

8. User Roles & Permissions

Nexus defines clear role-based access control (RBAC) with distinct permissions for external and internal users.

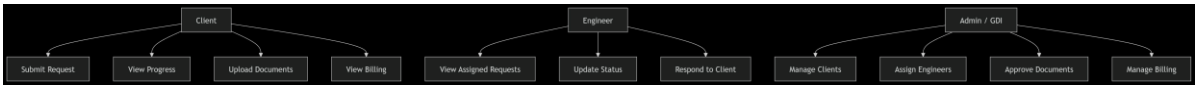

Figure 7 — User Role & Permissions Map

8.1 External Roles

Visitor (No Login)

- Can interact with the AI intake agent
- Can view public information and service pages
- Cannot access any dashboard or project data

Prospect (Registered, Not Yet Qualified)

- Can submit requests and schedule meetings
- Can view their own submission history
- Cannot access client workspace features

Client (Qualified & Active)

- Submit requests and support tickets
- View project progress, milestones, and deliverables
- Upload and access documents
- View billing information and invoices

8.2 Internal Roles

Engineer

- View assigned requests
- Update request status and add resolution notes
- Respond to client within the ticketing system

Admin / GDI

- Manage all clients and their lifecycle states
- Assign engineers to requests and projects
- Approve documents and contracts
- Manage billing, invoices, and subscriptions
- Full system access including monitoring dashboards

8.3 Permission Matrix

Action	Visitor	Prospect	Client	Engineer	Admin
AI Intake	✓	✓	✓	—	—
Submit Request	—	✓	✓	—	✓

View Own Projects	—	—	✓	—	✓
View Assigned Requests	—	—	—	✓	✓
Update Request Status	—	—	—	✓	✓
Manage Clients	—	—	—	—	✓
Manage Billing	—	—	—	—	✓
Approve Documents	—	—	—	—	✓
View Monitoring	—	—	✓*	✓	✓

** Clients see limited health status only (healthy/warning/critical). Full monitoring details are internal-only.*

9. Core Modules Reference

This section provides a quick reference for each module's scope and key data entities. Use this as a checklist during development.

9.1 AI Intake Module

- **Function:** Chatbot interface for initial discovery
- **Key outputs:** Structured problem summary, service classification, preliminary scope
- **Tech notes:** Uses GDI AI Agent (Claude API). Conversation stored for lead context.

9.2 Lead Management Module

- **Function:** Database of all incoming requests with qualification workflow
- **Key fields:** Company profile, problem description, service category, urgency, budget, timeline
- **Statuses:** New → Reviewing → Qualified / Rejected / Deferred

9.3 Scheduling Module

- **Function:** Calendar integration and meeting booking
- **Features:** Availability display, automatic confirmation, meeting type selection, notification dispatch
- **Integration:** Google Calendar API or Outlook Calendar API

9.4 Client Workspace Module

- **Function:** Central dashboard for active clients
- **Includes:** Project list, milestone tracker, task board, deliverable downloads, document repository

9.5 Ticketing System

- **Function:** Structured support request management
- **Categories:** Bug, access issue, feature request, maintenance, billing
- **Priority:** Low / Medium / High / Critical
- **Status flow:** Open → Review → In Progress → Waiting → Resolved → Closed

9.6 Contract Module

- **Function:** Digital agreement management
- **Features:** Template library, version control, scope/terms preview, e-signature (DocuSign/Adobe)

9.7 Billing Module

- **Function:** Invoice and subscription management
- **Features:** Invoice generation, payment status tracking, subscription plan management
- **Integration:** Stripe or local gateway (e.g., Midtrans)

9.8 Monitoring Module (Phase 3)

- **Client-facing:** System health status, uptime, storage, backup status, incident history, SSL/domain status
- **Internal-only:** Server logs, infrastructure controls, deployment pipelines, CPU/RAM details

10. Client Types

All client types share the same platform but receive different access scopes and billing models.

Type	Description	Billing	Dashboard Access
Project Client	One-time engagement for a specific deliverable	One-time payment	Active during project, read-only after
Maintenance Client	Ongoing support for a deployed system	Monthly retainer	Permanent with ticketing access
Subscription Client	Cloud-hosted system with full managed services	Recurring subscription	Full access including monitoring

11. Phased Development Plan

Nexus will be built incrementally across four phases. Each phase delivers a functional system that provides immediate value while laying the groundwork for the next.

Phase 1 — MVP: Structured Lead Acquisition

- **Scope:** AI intake form, scheduling system, basic lead management
- **Goal:** Replace ad-hoc lead handling with a structured pipeline
- **Deliverables:** Public-facing intake page, calendar booking, lead database with qualification statuses

Phase 2 — Client Operations

- **Scope:** Client workspace, project tracking, ticketing system
- **Goal:** Manage real client projects within the platform
- **Deliverables:** Authenticated dashboard, project/milestone views, support ticket flow

Phase 3 — Deal Closure

- **Scope:** Contract module, billing integration
- **Goal:** Close and manage deals entirely within Nexus
- **Deliverables:** Contract templates with e-signature, invoice generation, payment tracking

Phase 4 — Scale & Differentiate

- **Scope:** Infrastructure monitoring dashboard, AI enhancements, analytics
- **Goal:** Full platform maturity with monitoring and data-driven insights
- **Deliverables:** Client-facing health dashboards, internal analytics, advanced AI intake features

Phase	Key Modules	Priority
Phase 1	AI Intake, Scheduling, Lead Management	Immediate
Phase 2	Workspace, Projects, Ticketing	After Phase 1 launch
Phase 3	Contracts, Billing	After first active clients
Phase 4	Monitoring, Analytics, AI v2	Post-stabilization

12. Key Engineering Guidelines

12.1 General Principles

- Build for real client use cases first. Avoid speculative features.
- Keep the data model clean. One client identity across all modules.
- Enforce state transitions at the API layer, not just the UI.
- All inter-module communication goes through the API layer.
- Write integration tests for state transitions and notification triggers.

12.2 API Design

- RESTful endpoints grouped by module (e.g., /api/requests, /api/projects, /api/billing)
- Consistent error response format across all endpoints
- Rate limiting on public-facing endpoints (AI intake, scheduling)
- JWT-based authentication with role-scoped access tokens

12.3 Database Conventions

- Use UUID primary keys for all entities
- Include created_at, updated_at, and created_by on all tables
- Soft-delete pattern for client and project records (never hard-delete)
- State history table for audit trail on all state transitions

12.4 Notification Conventions

- All notifications must be idempotent (safe to retry)
- Template IDs stored in database, not hardcoded
- Delivery status logged for every attempt
- WhatsApp templates must be pre-approved via Meta Business Manager

---

Nexus is the foundation for GDI's evolution into a platform-driven technology partner. Build it right, build it incrementally, and keep the client experience at the center of every decision.